

Neural Networks with Möbius Basel-Check Memory

Unified Framework for Spectral Compression and Invariant Validation

Abstract

We present a unified framework integrating neural networks with a memory architecture based on the Möbius function and the Basel checksum. The memory stores coefficients at odd square-free indices, compresses signals via the chip pipeline (convolution + supertrace + Möbius compression), and uses the Basel sum

$$\sum_{n=1}^{\infty} \frac{\mu(n)}{n^2} = \frac{6}{\pi^2}$$

as an invariant to detect drift, prevent infinite loops, and bound network atrophy. The natural step $a = 1/(\pi - e) \approx 2.362$ emerges from the identity $a^{-1} + e = \pi$ and serves as the derivative step and error tolerance. All operations run in $O(K \log K)$ time.

1 Introduction

The combination of number theory and neural computation offers a novel approach to memory management and signal processing. The Möbius function $\mu(n)$, defined as

$$\mu(n) = \begin{cases} 1 & \text{if } n \text{ is square-free with an even number of prime factors,} \\ -1 & \text{if } n \text{ is square-free with an odd number of prime factors,} \\ 0 & \text{if } n \text{ has a squared prime factor,} \end{cases}$$

provides a natural filter for spectral coefficients. The Basel sum

$$\sum_{n=1}^{\infty} \frac{\mu(n)}{n^2} = \frac{6}{\pi^2}$$

gives the density of square-free numbers and acts as a checksum for memory integrity.

2 Mathematical Framework

2.1 Möbius Sieve

The linear sieve computes $\mu(1), \dots, \mu(K)$ in $O(K)$ time:

$$\begin{aligned} \mu(1) &= 1, \\ \mu(n) &= - \sum_{d|n, d < n} \mu(d) \quad \text{or via the linear sieve.} \end{aligned}$$

2.2 Harmonic Numbers

The harmonic numbers

$$H_n = \sum_{k=1}^n \frac{1}{k}$$

serve as the discrete time coordinate for the spectral sum $\zeta(t)$:

$$\zeta(t) = \sum_{i=-K}^K \mu(|i|) \exp\left(\frac{i t i}{\alpha}\right), \quad \alpha = \frac{1}{\pi - e}.$$

2.3 Supertrace and Entropy

For a signal C of length N , the supertrace is

$$S = \sum_{k=1}^N (-1)^{k+1} |C_k|,$$

the entropy is

$$H = -\alpha \frac{|S|}{N} \log\left(\frac{|S|}{N}\right),$$

and the invariant mass is

$$m = |S|e^{-H}.$$

The natural step $a = 1/(\pi - e)$ sets the error tolerance:

$$\Delta S \leq a \cdot N.$$

2.4 Basel Checksum

The Basel checksum is defined as

$$B(K) = \sum_{n=1}^K \frac{\mu(n)}{n^2}.$$

For large K , $B(K) \rightarrow 6/\pi^2$. The density of non-zero Möbius values is

$$\lim_{K \rightarrow \infty} \frac{|\{n \leq K : \mu(n) \neq 0\}|}{K} = \frac{6}{\pi^2}.$$

This serves as an invariant check for memory compression.

3 Memory Architecture

3.1 MobiusCollatzMemory

The memory stores key-value pairs (n, v) where n is odd and square-free ($\mu(n) \neq 0$). The Collatz step transforms indices:

$$n \mapsto \frac{3n+1}{2^k}, \quad k = v_2(3n+1),$$

keeping only odd square-free results.

3.2 Chip Compression Pipeline

The chip pipeline consists of:

1. TSP routing (bucket sort by phase)
2. Exponential convolution: $\text{conv} = (1 - \exp(-\alpha|i - j|))/\text{NORM}$
3. Supertrace and mass computation
4. Möbius compression: keep top $M = \max(1, \lfloor |S| \rfloor)$ square-free coefficients

All steps are $O(K \log K)$ due to sorting.

4 Neural Network Integration

4.1 Inference Buffer

The buffer holds a signal array and provides:

- `apply_gate(gate_name)`: log, exp, sin, cos, derivative
- `compress()`: chip compression
- `collatz()`: Collatz index transform
- `inverse_score_with(target)`: inversion count ($O(K \log K)$)

4.2 Atrophy Prevention

The inverse score between current activations and a reference is monitored. When it exceeds the threshold 0.3, a correction is applied:

$$\mathcal{L}_{\text{corr}} = \text{MSE}(h_{\text{cur}}, h_{\text{ref}}) \cdot \min(1, H_{\text{ent}}/0.5).$$

This mimics biological homeostasis.

4.3 Derivative Gate

The derivative operator

$$D_a f(t) = \frac{f(t+a) - f(t)}{a}, \quad a = \frac{1}{\pi - e},$$

is implemented as a logic gate, matching the finite-difference method from the Ruby code.

5 Compression and Serialization

5.1 Ternary Packing

Each Möbius value is packed into 2 bits:

$\mu(n)$	Code
0	00
1	01
-1	10

The compressed size is $\lceil K/4 \rceil$ bytes.

5.2 Basel Validation

After unpacking, the Basel sum is recomputed. If it deviates from $6/\pi^2$ beyond a tolerance, the memory is flagged as corrupted.

6 Convergence and Bounded Loops

The `MobiusConvergence` class applies gates, compression, and Collatz steps iteratively until the signal norm change falls below `tol` or `max_iter` is reached. The loop is bounded by `max_iter` $\times O(K \log K)$.

7 Applications

- **Text processing:** ASCII signals are compressed via the Möbius gate.
- **Collatz reinforcement learning:** SuperTrace reward + inverse-score match.
- **Cross-compilation:** Code lines are hashed into spectral coefficients.
- **Harmonic oscillator simulation:** Step gates enumerate the oscillator in $O(K \log K)$.

8 Conclusion

The Möbius Basel-check memory provides a robust, number-theoretic foundation for neural network memory management. The natural constant $a = 1/(\pi - e)$ unifies the derivative step, entropy scaling, and error tolerance. The Basel checksum ensures integrity, prevents infinite loops, and bounds drift. All operations are $O(K \log K)$, making the framework scalable and practical.

References

1. Möbius function and the Basel problem: `basel-check-error.py`
2. Spectral sum and harmonic points: `harmonic-alpha-0k.py`

3. Chip compression pipeline: `chip-g.py`, `mobius-gates.py`
4. Neural atrophy prevention: `atrophy.py`
5. Derivative operator: `derida.rb`
6. Collatz memory: `random-access-colla-mobius.py`